Task 3 — PID Control for Line Following

1. What is PID control?

PID stands for Proportional, Integral and Derivative. It is a control method used to help a robot follow a line by continuously adjusting the speed of its left and right motors.
In your robot, five IR sensors detect the line. The controller calculates how far the robot is from the line's centre and adjusts the motors to correct its direction.

2. Role of P, I and D terms

i) Proportional — reacts to the current error

The larger the current error, the larger the correction. If the robot moves away from the line's centre, P control adjusts the motors to steer it back.

- Advantage: simple and responds quickly.
- Limitation: too much P gain can cause zigzagging or oscillation.

ii) Integral — reacts to accumulated error

It adds up the error over time. It helps correct a persistent error that P control alone may not eliminate.

- Advantage: can reduce steady-state error.
- Limitation: too much integral action can cause overshoot and oscillation.

iii) Derivative — reacts to how quickly the error changes

It measures the change in error over time and helps oppose rapid changes. This can reduce overshoot and make steering smoother.

- Advantage: helps reduce oscillation.
- Limitation: sensor noise can make the derivative term unstable.

The general PID equation is:

$$u = K_P e + K_I \int e dt + K_D \frac{de}{dt}$$

Here, e is the line-position error, u is the motor correction, and K_P, K_I, and K_D are the controller gains.

3. Which configuration would I choose?

I would choose PD control (Proportional + Derivative) for this line-following robot.

Reasons:

- P responds to the robot's current position error.
- D helps reduce sudden steering changes and zigzagging.
- I is often unnecessary when a line-following robot can track the line effectively using PD control.
- PD is simpler to tune than full PID and avoids integral windup.

I would begin with a low P gain, increase it until the robot follows the line reasonably well, and then adjust D to reduce oscillation. If testing reveals a persistent error that PD cannot correct, I would consider adding a small I term.

4. Converting sensor readings into error

Robot has five sensors with these weights:

Sensor	Position	Weight
S1	Far Left	-2
S2	Left	-1
S3	Centre	0
S4	Right	1
S5	Far Right	2

The sensor readings are combined using a weighted average to estimate the line's position. This follows the standard weighted-centroid approach used in line-following controllers.

$$Position \ = \ \Sigma(Reading\ X\ weight) \ \div \ \Sigma(Reading)$$

The target position is zero, so:

$$Error = Position - 0$$

For example, suppose the sensor readings are:

$$[0,\ 500,\ 1000,\ 500,\ 0]$$

Then:

$$Position = \frac{(0)(-2)+(500)(-1)+(1000)(0)+(500)(1)+(0)(2)}{0+500+1000+500+0} = 0$$

The error is zero, meaning the line is centred under the sensor array.

- Negative error: line is toward the left.
- Zero error: line is centred.
- Positive error: line is toward the right.

This sign convention assumes higher readings mean stronger detection of the line, as specified for your sensors.

5. Converting error into motor commands

The PD controller calculates the correction:

$$Correction = K_P e + K_D \frac{e - e_{previous}}{\Delta t}$$

Then the correction is applied to the motor speeds:

L = Base Speed + Correction

R = Base Speed - Correction

Here, L is the left motor command and R is the right motor command. Depending on your motor wiring and the error sign convention, you may need to reverse the correction signs.

Finally, clamp the motor commands to the permitted range so that the output stays within the motor driver's limits.

6. Responding to oscillation or a lost line

If the robot oscillates or zigzags:

- Reduce K_P if the robot makes overly strong corrections.
- Adjust K_D to dampen rapid changes in error.
- Reduce the base speed while tuning the controller.
- Filter noisy sensor readings if necessary.

If the robot loses the line:

- Detect when the total sensor reading is too low to calculate a reliable position.
- Stop both motors instead of using an invalid position.
- Optionally, use the last known error to perform a controlled search for the line, with a timeout before stopping completely.